

## 2.3 Producing robust programs

Where this unit sits, what it covers, and how far

### Where this unit sits

|                                    |                                                                                                                                                                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Qualification                      | GCSE Computer Science (9-1), specification code J277, from 2020                                                                                                                                                                              |
| Component                          | Component 02: Computational thinking, algorithms and programming — 80 marks, 1 hour 30 minutes                                                                                                                                               |
| What OCR says the component covers | computational thinking: algorithms, programming techniques, producing robust programs, computational logic and translators                                                                                                                   |
| This unit                          | topic 2.3 Producing robust programs                                                                                                                                                                                                          |
| Checked                            | 3 October 2026, at <a href="https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/">https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/</a> |

**How far this has been checked.** The topic numbers and titles below are the ones this course uses on its worksheets and slides. They were not re-checked against the published specification document when this map was built: the document could not be retrieved from this build environment. Check them against <https://www.ocr.org.uk/Images/558027-specification-gcse-computer-science-j277.pdf> before relying on this map for a scheme of work, and treat any finer subdivision as absent rather than assumed - no sub-point numbers are claimed anywhere in these documents, because none were verified.

**Before it:** 1.1 Systems architecture, 1.2 Memory and storage, 1.3 Computer networks, connections and protocols, 1.4 Network security, 1.5 Systems software, 1.6 Ethical, legal, cultural and environmental impacts of digital technology, 2.1 Algorithms, 2.2 Programming fundamentals

**After it:** 2.4 Boolean logic, 2.5 Programming languages and Integrated Development Environments, PY Python programming

### The lesson sequence and the questions it answers

| # | LESSON                                  | THE BIG QUESTION                                                                 | ACTIVITIES | MARKS |
|---|-----------------------------------------|----------------------------------------------------------------------------------|------------|-------|
| 1 | <b>Security desk</b><br>rp-l01          | How can a programmer make sure a program copes with anything a user types?       | 13         | 19    |
| 2 | <b>Cyber defence HQ</b><br>rp-l02       | How can a program defend itself against careless users and deliberate attackers? | 13         | 20    |
| 3 | <b>Code clinic</b><br>rp-l03            | What does code maintainability mean?                                             | 14         | 24    |
| 4 | <b>Bug hunt lab</b><br>rp-l04           | What are the different types of errors that can occur in a program?              | 13         | 20    |
| 5 | <b>Test lab</b><br>rp-l05               | What makes a good testing strategy?                                              | 14         | 23    |
| 6 | <b>Revision HQ 2.3</b><br>rp-l06        | What makes a robust program?                                                     | 18         | 23    |
| 7 | <b>End-of-topic test</b><br>rp-l07-test | assesses the whole unit                                                          | 0          | 0     |

## Prior knowledge, vocabulary and skills

**Assumed before the unit:** the outcomes of 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2.1, 2.2

**Vocabulary introduced:** Defensive design, Anticipating misuse, Authentication, Input validation, Maintainability, Naming conventions, Testing, Iterative testing, Logical error, Test data

**What a pupil does, not only reads:** code, match, mcq, multi, sort

## Coverage matrix

Every outcome this unit states, and whether it is taught, practised and assessed **within this unit**. Full means all three; partial means some; not covered means none of the three could be traced.

**How this was worked out.** The trace is derived by matching the content words of an outcome against the station text, the activities and the exam questions of the unit, not authored by hand. The words each match was made on are in `build/coverage.json`. It shows where to look; it does not certify that the alignment is right, and a full row has not been checked by a person either.

| OUTCOME                                                                                                | LESSON | TAUGHT | PRACTISED | ASSESSED | COVERAGE |
|--------------------------------------------------------------------------------------------------------|--------|--------|-----------|----------|----------|
| know what is meant by defensive design when writing programs<br><code>rp-l01-o1</code>                 | rp-l01 | yes    | yes       | yes      | Full     |
| understand why input validation is necessary<br><code>rp-l01-o2</code>                                 | rp-l01 | yes    | yes       | yes      | Full     |
| know a range of validation techniques used to write robust programs<br><code>rp-l01-o3</code>          | rp-l01 | yes    | yes       | yes      | Full     |
| know what is meant by defensive design considerations<br><code>rp-l02-o1</code>                        | rp-l02 | —      | —         | yes      | Partial  |
| know a range of problems that can occur when a program is running<br><code>rp-l02-o2</code>            | rp-l02 | yes    | yes       | —        | Partial  |
| understand some authentication techniques programmers use<br><code>rp-l02-o3</code>                    | rp-l02 | yes    | yes       | yes      | Full     |
| know why creating code that is easy to read is important with large projects<br><code>rp-l03-o1</code> | rp-l03 | yes    | yes       | —        | Partial  |
| understand what programmers can do to make their code more readable<br><code>rp-l03-o2</code>          | rp-l03 | yes    | yes       | yes      | Full     |
| understand how to refine algorithms to make them more robust<br><code>rp-l03-o3</code>                 | rp-l03 | yes    | yes       | yes      | Full     |
| know four reasons why a program should be tested<br><code>rp-l04-o1</code>                             | rp-l04 | yes    | yes       | yes      | Full     |
| know what iterative testing is<br><code>rp-l04-o2</code>                                               | rp-l04 | yes    | yes       | yes      | Full     |
| know what final/terminal testing is<br><code>rp-l04-o3</code>                                          | rp-l04 | yes    | yes       | yes      | Full     |
| know what a syntax error is<br><code>rp-l04-o4</code>                                                  | rp-l04 | yes    | yes       | yes      | Full     |

| OUTCOME                                                                                                   | LESSON | TAUGHT | PRACTISED | ASSESSED | COVERAGE |
|-----------------------------------------------------------------------------------------------------------|--------|--------|-----------|----------|----------|
| know what a logic error is<br><b>rp-104-o5</b>                                                            | rp-104 | yes    | yes       | yes      | Full     |
| understand that just because a program works doesn't mean it works for all inputs<br><b>rp-105-o1</b>     | rp-105 | yes    | yes       | yes      | Full     |
| understand that test data should include normal, boundary, invalid and erroneous data<br><b>rp-105-o2</b> | rp-105 | yes    | yes       | yes      | Full     |
| understand how robust programs are made<br><b>rp-106-o1</b>                                               | rp-106 | yes    | yes       | yes      | Full     |
| know which lessons I'm secure on and which to revise<br><b>rp-106-o2</b>                                  | rp-106 | yes    | yes       | —        | Partial  |

**14 full** · **4 partial** · **0 not covered**, of 18 outcomes.

## Gaps, assumptions and overlap

- **rp-102-o1** know what is meant by defensive design considerations — no station's text matches it
- **rp-102-o2** know a range of problems that can occur when a program is running — nothing in this unit assesses it
- **rp-103-o1** know why creating code that is easy to read is important with large projects — nothing in this unit assesses it

### Assessed, but matching no outcome this unit states

Each of these is a real exam question on a worksheet in this unit. Either the outcome it tests is worded differently, or it is genuinely assessing something the unit does not claim to teach.

- rp-101-x2 A form asks for a UK mobile number. Describe two validation checks it should make, and say what each one catches.
- rp-101-x3 Explain the difference between a range check and a type check, using an example of each.
- rp-103-x2 Rewrite the first five lines of Exhibit A (on the floor of the clinic) using good practice.
- rp-107-test-x1 Maintainability: names, comments, indentation, sub-programs
- rp-107-test-x3 Authentication and anticipating misuse

### What could not be traced

The wall text of rp-103, rp-104 is painted into the 360 image and held nowhere as text, so an outcome in those lessons lesson can only be traced through its activities and its exam questions. Read the rows above for them with that in mind.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.